

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Вычислительная математика»
Тема: Метод простых итераций

Студент гр. 4342

Садыков С.Р.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Нахождение корня уравнения методом простых итераций с заданной точностью Eps , исследование зависимости числа итераций от точности Eps при заданном изменении Eps , исследование чувствительности к ошибкам в исходных данных.

Задание.

Используя программы-функции ITER и Round из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения $f(x)=0$ с заданной точностью Eps методом итераций, исследовать скорость сходимости и обусловленность метода.

Вариант 20: $f(x) = e^x - \arccos(\sqrt{|x|})$

Порядок выполнения лабораторной работы №6:

- 1) Графически или аналитически отделить корень уравнения $f(x)=0$.
- 2) Преобразовать уравнение $f(x)=0$ к виду $x=j(x)$ так, чтобы в некоторой окрестности $[\text{Left}, \text{Right}]$ корня производная $j'(x)$ удовлетворяла условию $j'(x) \leq q < 1$. При этом следует иметь в виду, что чем меньше величина q , тем быстрее последовательные приближения сходятся к корню.
- 3) Выбрать начальное приближение, лежащее на $[\text{Left}, \text{Right}]$.
- 4) Составить подпрограмму для вычисления значений $j(x), j'(x)$, предусмотрев округление вычисленных значений с точностью Delta .

Теоретические положения.

Метод простых итераций решения уравнения $f(x)=0$ состоит в замене исходного уравнения эквивалентным ему уравнением $x = j(x)$ и построении последовательности $x_{n+1} = j(x_n)$, сходящейся при $n \rightarrow \infty$ к точному решению. Достаточные условия сходимости метода простых итераций формулируются теоремой, приведенной [1,2,7].

Рассмотрим один шаг итерационного процесса. Исходя из найденного на предыдущем шаге значения x_{n-1} , вычисляется $y = j(x_{n-1})$. Если $|y - x_{n-1}| > \epsilon$, то полагается $x_n = y$ и выполняется очередная итерация. Если же $|y - x_{n-1}| < \epsilon$, то вычисления заканчиваются и за приближенное значение корня принимается величина $x_n = y$. Погрешность результата вычислений зависит от знака производной $j'(x)$: при $j'(x) > 0$ погрешность определения корня составляет $q1q$, а при $j'(x) < 0$ погрешность не превышает q . Здесь q - число, такое, что $|j'(x)| \leq q$ на отрезке $[a,b]$. Существование числа q является условием сходимости метода в соответствии с отмеченной выше теоремой.

Для применения метода простых итераций определяющее значение имеет выбор функции $j(x)$ в уравнении $x = j(x)$, эквивалентном исходному. Функцию $j(x)$ необходимо подбирать так, чтобы $|j'(x)| \leq q < 1$. Это обуславливается тем, что если $j'(x) < 0$ на отрезке $[a,b]$, то последовательные приближения $x_n = j(x_{n-1})$ будут колебаться около корня s , если же $j'(x) > 0$, то последовательные приближения будут сходиться к корню s монотонно. Следует также помнить, что скорость сходимости последовательности $\{x_n\}$ к корню s функции $f(x)$ тем выше, чем выше число q .

Выполнение работы.

1) Корень уравнения $f(x) = 0$ для $f(x) = e^x - \arccos(\sqrt{x})$ - был найден графически при помощи графического калькулятора Desmos. Полученный график представлен на рисунке 1.

Рассмотрим отрезок $[0.154405, 0.154407]$. Функция монотонно возрастающая. На данном отрезке функция непрерывна и монотонно возрастает, а на концах отрезка принимает значения разных знаков ($f(0.154405) < 0$, $f(0.154407) > 0$), что подтверждает наличие единственного корня. Для реализации метода простых итераций выразим x через $\arccos$:
 $e^x = \arccos(\sqrt{x}) \rightarrow \sqrt{x} = \cos(e^x) \rightarrow x = \cos^2(e^x)$

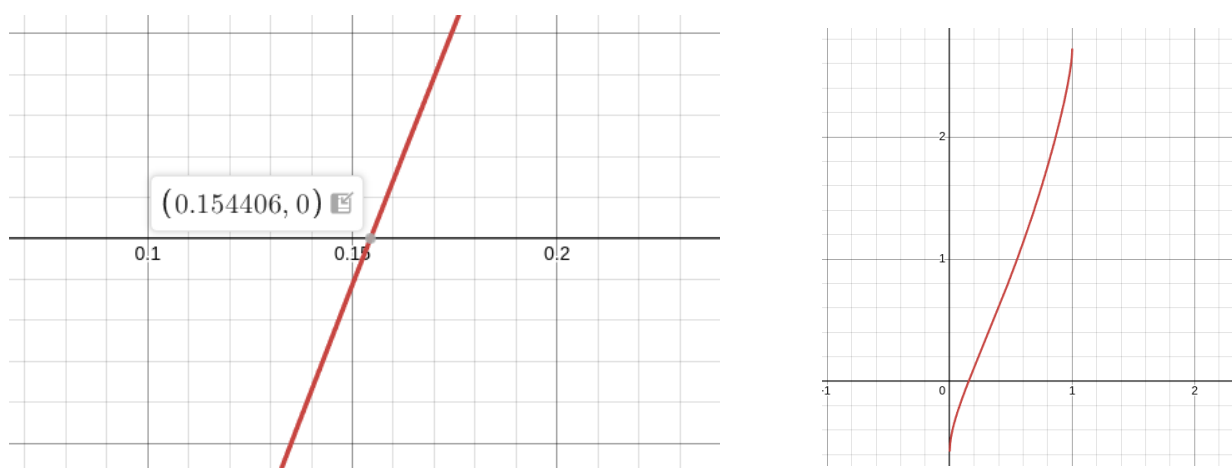


Рисунок 1 — График функции $f(x) = e^x - \arccos(\sqrt{x})$

Производная выраженной функции $(-\sin(2e^x)e^x)$ в районе $x = 0.154$: $0.836 < 1$, следовательно, метод будет сходиться.

2) Для применения метода уравнение $f(x) = 0$ было преобразовано к виду $x = \phi(x)$, где $\phi(x) = \cos^2(e^x)$. Данная функция возвращает вычисленное значение $\phi(x)$. Глобальная переменная `CurrentDelta` отвечает за значение погрешности: если она больше нуля, результат $\phi(x)$ округляется при помощи функции `Round` с заданной точностью.

3) В функции `main` задаётся начальное значение $x_0 = 0.154405$. Данные значения удовлетворяют условию сходимости.

4) Затем в цикле выполняется функция `ITER` для значений точности `Eps` в диапазоне от 0.1 до 0.000001. Полученные результаты выводятся в поток вывода и передаются скрипту `plots.py`. В данном файле с помощью библиотеки `matplotlib` строится график зависимости числа итераций n от точности `Eps`. Для наглядности по оси абсцисс используется логарифмическая шкала. Из полученного графика видно, что количество итераций растёт линейно при увеличении точности (уменьшении `Eps`), что экспериментально подтверждает линейную скорость сходимости метода простых итераций. В сравнении с методом Ньютона, данный метод требует значительно большего числа шагов для достижения аналогичной точности.

Сами измерения представлены в таблице 1.

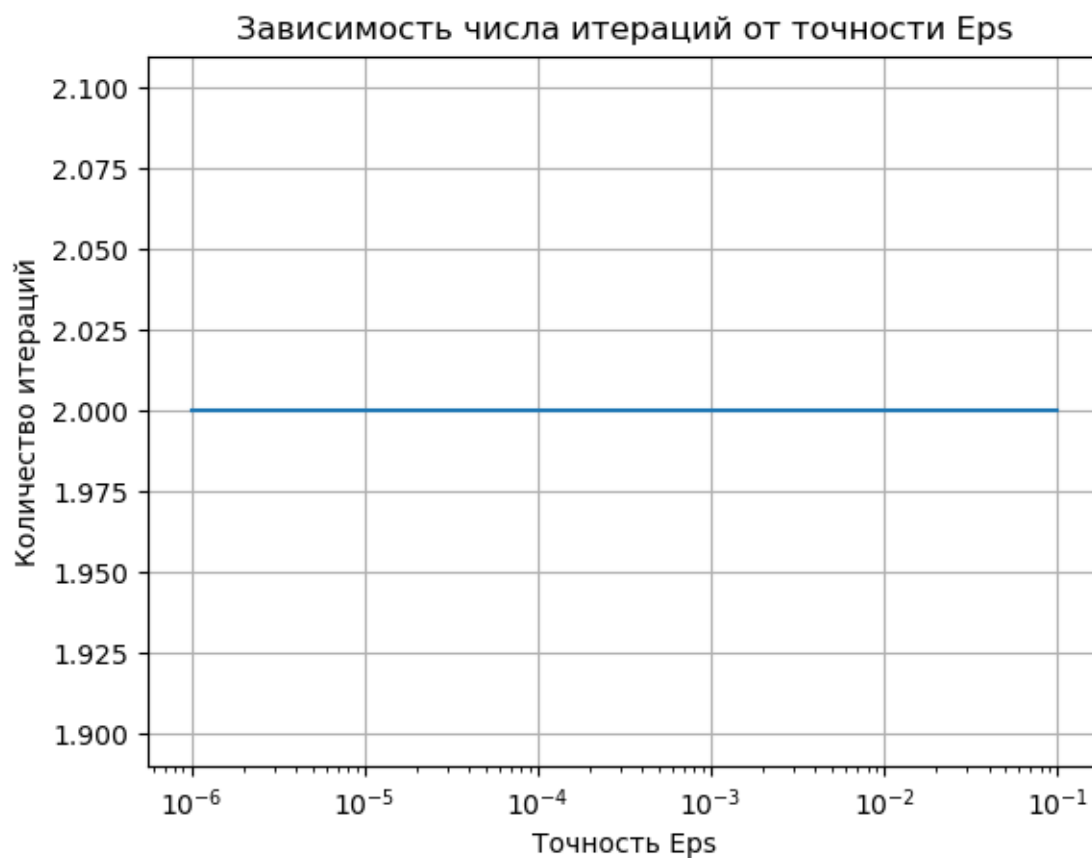


Рисунок 2 — График зависимости числа итераций от точности Eps

Таблица 1 — Исследование зависимости числа итераций от точности Eps

Точность Eps	Число итераций	Приблизженный корень
0.1	2	0.1544052223
0.01	2	0.1544052223
0.001	2	0.1544052223
0.0001	2	0.1544052223
0.00001	2	0.1544052223
0.000001	2	0.1544052223

5) Далее было произведено исследование чувствительности метода к ошибкам в исходных данных. Для этого была заведена глобальная переменная `CurrentDelta`, отвечающая за значение погрешности. Если она больше нуля, то в функции `RNI` вычисленное значение итерационной функции $\phi(x)$ округляется при помощи функции `Round` с заданной точностью. В функции `main()` в цикле выполняется функция `ITER` для погрешностей `Delta` от 10^{-3} до 10^{-8} . Результаты измерений числа итераций и вычисленного корня подаются на вход файлу `plots.py`, в котором строится график зависимости вычисленного приближенного корня от погрешности `Delta` (см. рис. 3). Шкалы — линейные. Из графика и полученных данных видно, что при уменьшении погрешности `Delta` вычисленное значение стремится к истинному корню. Однако при грубых значениях погрешности (порядка 10^{-3}) наблюдается потеря устойчивости метода (зацикливание), что связано с тем, что величина округления становится соизмеримой с шагом итерации. Полученные измерения представлены в таблице 2.

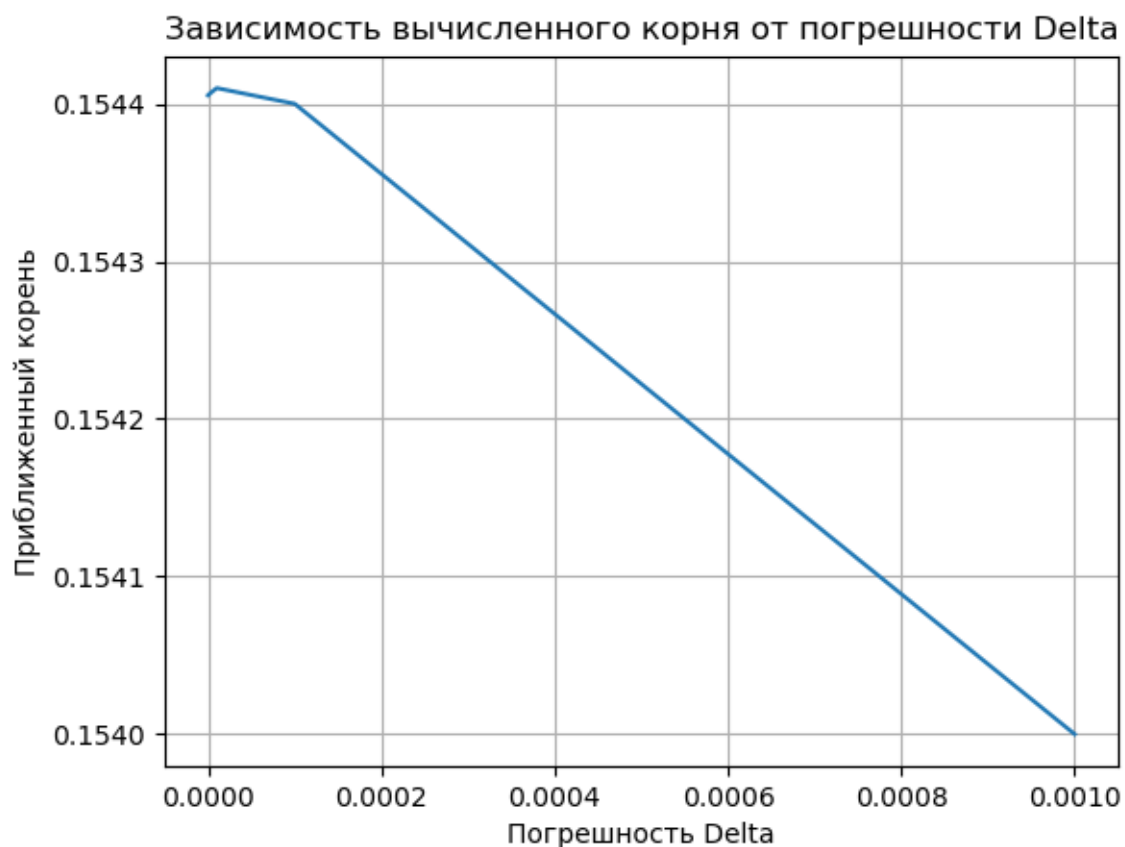


Рисунок 3 — Зависимость вычисленного корня от погрешности Delta

Таблица 2 — Исследование чувствительности к ошибкам исходных данных

Погрешность Delta	Число итераций	Приблизженный корень
0.001	1001	0.154
0.0001	2	0.1544
0.00001	1001	0.15441
0.000001	2	0.154406
0.0000001	2	0.1544052
0.00000001	2	0.15440522

Разработанный программный код см. в приложении А.

Выводы.

В ходе лабораторной работы был найден корень уравнения методом простых итераций с заданной точностью Eps . Вычисленный результат совпал с результатом, вычисленным при помощи графического калькулятора. Было произведено исследование зависимости числа итераций от точности Eps при заданном изменении Eps , а также исследование чувствительности к ошибкам в исходных данных.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb6.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

// Глобальная переменная для точности округления
double CurrentDelta = 0.0;

// Прототипы функций
double Round(double X, double Delta);
double PHI(double x); // Переименовали для ясности, так как это x =
phi(x)
double ITER(double X0, double Eps, int &N);

// Функция округления
double Round(double X, double Delta) {
    if (Delta <= 1E-9) return X;
    if (X > 0.0) return (Delta * (long((X / Delta) + 0.5)));
    else return (Delta * (long((X / Delta) - 0.5)));
}

// Итерационная функция phi(x) = cos^2(e^x)
double PHI(double x) {
    double raw = pow(cos(exp(x)), 2);
    if (CurrentDelta > 1e-9) return Round(raw, CurrentDelta);
    return raw;
}

// Метод простых итераций
double ITER(double X0, double Eps, int &N) {
    if (Eps <= 0.0) exit(1);

    double X1 = PHI(X0);
    double X2 = PHI(X1);
    N = 2;

    // Условие выхода
    while ((X1 - X2) * (X1 - X2) > fabs((2 * X1 - X0 - X2) * Eps))
    {
        X0 = X1;
        X1 = X2;
        X2 = PHI(X1);
        N++;
        if (N > 1000) break; // Защита от бесконечного цикла, если
        условие сжатия не выполнено
    }
    return X2;
}

int main() {
    double x0 = 0.154405; // Начальное приближение

    // --- БЛОК 1: Зависимость N от Eps ---
```

```
double EpsValues[] = {1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6};
int numEps = 6;
```

Python printf("%d\n", numEps); // Кол-во строк для первого цикла

```
CurrentDelta = 0.0;
for (int i = 0; i < numEps; i++) {
    int iter;
    double res = ITER(x0, EpsValues[i], iter);
    // Формат: eps n x
    printf("%.10f %d %.10f\n", EpsValues[i], iter, res);
}

// --- БЛОК 2: Чувствительность к Delta ---
double DeltaValues[] = {1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8};
int numDelta = 6;
double fixedEps = 1e-6;
```

Python printf("%d\n", numDelta); // Кол-во строк для второго цикла

```
for (int i = 0; i < numDelta; i++) {
    CurrentDelta = DeltaValues[i];
    int iter;
    double res = ITER(x0, fixedEps, iter);
    // Формат: delta n x
    printf("%.10f %d %.10f\n", DeltaValues[i], iter, res);
}
```

```
return 0;
```

```
}
```

Название файла: plots.py

```
from matplotlib import pyplot as plt
```

```
def plot(x_data, y_data, x_label, y_label, name, file_name,
is_log_xscale = False):
```

```
    plt.plot(x_data, y_data)
    plt.grid()
    if is_log_xscale:
        plt.xscale('log')
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.title(name)
    plt.savefig(file_name)
    plt.show()
```

```
def main():
    eps_values = []
    n_values = []
    delta_values = []
    x_values = []
```

eps"); print("Исследование зависимости числа операций от точности

```
numEps = int(input())
```

```

for _ in range(numEps):
    eps, n, x = [float(el) for el in input().split()]
    print(f"eps = {eps}, n = {n}, x = {x}")
    eps_values.append(eps)
    n_values.append(n)

    plot(eps_values, n_values, "Точность Eps", "Количество
итераций", "Зависимость числа итераций от точности Eps", "eps_n.png",
is_log_xscale=True)

    print("\nИсследование чувствительности к ошибкам исходных
данных")
    numDelta = int(input())
    for _ in range(numDelta):
        delta, n, x = [float(el) for el in input().split()]
        print(f"delta = {delta}, n = {n}, x = {x}")
        delta_values.append(delta)
        x_values.append(x)

        plot(delta_values, x_values, "Погрешность Delta", "Приближенный
корень", "Зависимость вычисленного корня от погрешности Delta",
"delta_x.png")

if __name__ == "__main__":
    main()

```